



Memoria de proyecto para *CiviLink*

27 de Abril de 2026

Miembros del equipo:

- ★ Alejandro Moreno
- ★ Alonso de Lombas
- ★ Daniel Maksymiv
- ★ Inés Triviño
- ★ Iván Fernández
- ★ Jacobo García-Lago
- ★ Javier Wohler Jaime
- ★ Marcos Gago
- ★ Mónica Martín
- ★ Sergio Zurrón



Índice:

1. Arquitectura del sistema.....	3
Capa de integración.....	3
Capa de negocio.....	4
Capa de presentación.....	4
2. Patrones de diseño aplicados.....	4
2.1 Controlador.....	4
2.2 Factoría abstracta.....	5
2.3 Singleton.....	5
2.4 Data Access Object (DAO).....	5
2.5 Transfer Objects (DTO).....	5
2.6 Application Service.....	5
3. Diagramas de clases.....	6
4. Diagramas de secuencia.....	6
5. Gestión de transacciones.....	6
6. Limitaciones del sistema y futuras mejoras.....	7
7. Repositorios del proyecto.....	7

1. Arquitectura del sistema

La aplicación CiviLink sigue una arquitectura común de tres capas. Esta estructura es ampliamente usada en sistemas empresariales por su capacidad de organización, escalabilidad y mantenibilidad. Es un enfoque que nos permite separar claramente las responsabilidades del sistema en diferentes capas y módulos que ayudan a desarrollar el software tanto en el ámbito colaborativo como a lo largo del tiempo. Estas características, hacen de la arquitectura multicapa la decisión evidente para CiviLink, que actúa como aplicación empresarial para la gestión de un centro cívico y de una cafetería, sistemas complejos que se benefician de un diseño escalable y modular.

Nuestro objetivo es lograr un bajo acoplamiento entre componentes. De esta forma, los cambios en una parte del sistema no afectan directamente a las otras capas, reduciendo el riesgo de errores y mejorando la reutilización de código.

1.1 Capa de integración

La capa de integración, también conocida como capa de acceso a datos, es la responsable de la persistencia de la información. Su principal función es gestionar la comunicación entre la aplicación y la base de datos, encapsulando toda la lógica necesaria para realizar operaciones de almacenamiento, consulta, actualización y eliminación de datos (también conocidas como operaciones CRUD).

Esta capa se implementa mediante el uso del patrón **DAO (Data Access Object)**, que abstrae completamente el acceso a la base de datos. Cada entidad del sistema dispone de su propio DAO, encargado de ejecutar operaciones CRUD mediante consultas SQL o mecanismos equivalentes, así como gestionar la conexión. Los DAOs se obtienen a través de una factoría correspondiente.

En el caso del módulo de gestión del centro cívico, se utiliza **Java Persistence API (JPA)** como tecnología de persistencia. JPA permite mapear objetos del dominio a tablas de la base de datos de forma transparente, reduciendo significativamente la cantidad de código necesario y facilitando el mantenimiento. Por lo tanto, la sección de centro cívico no cuenta con DAOs como tal (como se puede observar en el código), si no que los objetos de negocio son también objetos JPA gracias al uso de anotaciones.

Por otro lado, en el módulo de cafetería se emplea una versión más básica de persistencia (denominada en el proyecto como JPBD), que implica un mayor control manual sobre las consultas y la conexión a la base de datos, mediante la definición explícita de la capa de integración.

Usar ambas tecnologías en nuestro proyecto, nos ha sido de gran ayuda para entender las diferencias tanto conceptuales como prácticas entre ellos y así entender sus pros y sus contras.

Por último, esta capa es responsable de transformar los datos entre diferentes formatos. Concretamente, convierte los resultados de la base de datos en **DTOs (Data Transfer Objects)**, que son utilizados por las capas superiores.

1.2 Capa de negocio

La capa de negocio constituye el núcleo funcional del sistema. Es la encargada de implementar toda la lógica relacionada con las reglas del negocio, validaciones, procesos y operaciones que definen el comportamiento de la aplicación.

Esta capa se estructura mediante el uso de **SA (Service Applications)**, que actúa como intermediario entre la capa de presentación y la capa de integración. Cada SA representa un conjunto de métodos relacionados que forman una interfaz o fachada de operaciones de negocio.

Por lo tanto, valida los datos recibidos desde la interfaz de usuario (es la capa en la que se realizan las validaciones críticas de negocio y se manejan las excepciones y errores surgidos), aplica las reglas de negocio correspondientes, coordina los DAOs para casos en los que se requiere interacción con varias entidades, y gestiona las transacciones (por lo que la lógica de la concurrencia también se implementa en los SA).

Un aspecto importante de esta capa es que centraliza toda la lógica, evitando que esta se disperse en la interfaz gráfica o en los DAOs. Esto facilita enormemente el mantenimiento, las pruebas unitarias y la evolución del sistema.

Sin embargo esta capa no se encarga solo de las operaciones, si no que también es dónde se declaran los *transfers* / DTOs de las entidades, y sus objetos de negocio.

1.3 Capa de presentación

La capa de presentación es la encargada de la interacción con el usuario. Engloba la interfaz gráfica de la aplicación (GUI) con sus elementos visuales (ventanas, formularios, etc), así como un controlador para manejar los eventos de interacción (que determinan las llamadas a operaciones o la invocación de otra vista). Son los *actionListener* de la GUI los que toman nota de las interacciones interesantes para enviárselas al controlador.

Esta capa, por lo tanto, recoge la información introducida por el usuario, la valida a nivel básico, para después encapsularla en un *transfer* que envía a la capa de negocio.

En nuestra aplicación, cada pantalla de la aplicación es una clase distinta, que o bien pide datos, o muestra resultados de algún tipo (por ejemplo el listado de productos en la gestión de la cafetería).

2. Patrones de diseño aplicados

El diseño de la aplicación CiviLink se basa en la aplicación de diversos patrones de diseño clásicos, cuyo objetivo es proporcionar soluciones reutilizables a problemas comunes en el desarrollo de software. A continuación, se describen los principales patrones utilizados:

2.1 Controlador

El patrón Controlador actúa como el eje central de la aplicación. Se encarga de gestionar el flujo de ejecución, recibiendo eventos desde la interfaz gráfica y delegando las operaciones correspondientes en los Service Applications.

Este patrón permite desacoplar la lógica de presentación de la lógica de negocio, mejorando la modularidad del sistema.

Por ejemplo, en el caso de uso de “alta de usuario” (en el centro cívico), el controlador recibe el evento desde la interfaz, construye el DTO correspondiente con los datos proporcionados, y delega la operación en el SA de usuario.

2.2 Factoría abstracta

El patrón Factoría Abstracta se utiliza para la creación de objetos sin necesidad de especificar su clase concreta. En Civilink, se emplea sobre todo para instanciar DAOs dentro de las transacciones de las operaciones del SA.

Gracias a este patrón el código depende de interfaces en lugar de implementaciones concretas. Esto facilita también la sustitución de componentes (por ejemplo, cambiar una implementación de DAO sin afectar al resto del sistema).

2.3 Singleton

El patrón **Singleton** garantiza que ciertas clases tengan una única instancia en toda la aplicación. En este proyecto, se aplica principalmente al Controlador y al EntityManager. Este enfoque nos permite reducir el consumo de memoria y el coste de inicialización de clases complejas. Además, es especialmente relevante en el contexto de las operaciones concurrentes de nuestra aplicación, donde es importante mantener consistencia en el acceso a los recursos.

2.4 Data Access Object (DAO)

Como hemos descrito anteriormente, el patrón DAO encapsula el acceso a los datos, proporcionando una interfaz clara para interactuar con la base de datos.

Cada DAO implementa operaciones específicas para su entidad correspondiente, delegando en la base de datos las tareas de almacenamiento.

2.5 Transfer Objects (DTO)

Los DTOs son objetos simples que transportan datos entre capas. Se caracterizan por no contener lógica de negocio, únicamente atributos y métodos de acceso. Nos permiten reducir las llamadas entre capas (mayor modularidad) y evitar la exposición directa de entidades internas que podrían ser privadas.

2.6 Application Service

Como hemos mencionado anteriormente, el patrón SA sirve como fachada para las operaciones de negocio de la aplicación. El controlador llama a un método abstracto, mientras que la implementación se realiza internamente, desacoplando la presentación (llama a un método, no le importa cómo está implementado) de la implementación (que se puede editar y cambiar en el futuro sin tener que cambiar nada en la capa de presentación).

3. Diagramas de clases

En el repositorio de modelado del proyecto se incluyen diagramas de clases detallados para cada entidad del sistema. Estos diagramas representan la estructura estática del sistema, mostrando:

- Clases y atributos.
- Métodos y responsabilidades.
- Relaciones entre clases (asociaciones, herencias, dependencias).

4. Diagramas de secuencia

Los diagramas de secuencia disponibles en el repositorio representan el comportamiento dinámico del sistema.

Para cada entidad del sistema, se ha modelado al menos un caso de uso representativo, incluyendo:

- Interacciones desde la interfaz (ActionListeners).
- Invocaciones a los Service Applications.
- Acceso a los DAOs.

Estos diagramas ayudan a comprender el flujo de ejecución y son especialmente útiles para validar el diseño antes de su implementación.

5. Gestión de transacciones

La gestión de transacciones en CiviLink constituye un aspecto clave para garantizar la consistencia e integridad de los datos, especialmente en aquellas operaciones que implican múltiples pasos o afectan a varias entidades del sistema. En este sentido, se ha adoptado un enfoque en el que los SAs actúan como frontera transaccional, encapsulando operaciones complejas dentro de una única unidad lógica de ejecución.

Las operaciones son atómicas, es decir, deben completarse en su totalidad o no ejecutarse en absoluto. Un ejemplo representativo es el proceso de cierre de una venta en la cafetería, donde intervienen distintas acciones como la validación del pedido, el cálculo del importe total, el almacenamiento del ticket y la actualización del stock de productos. En este contexto, si alguna de estas fases falla (por ejemplo, por falta de stock o error en la persistencia), el sistema evita estados intermedios inconsistentes, abortando la operación completa.

Se aprovechan los mecanismos aprendidos en clase para la declaración de clases (como *Transaction*) que simplifican el control transaccional concurrente, delegando en ellas la gestión de commits y rollbacks.

6. Limitaciones del sistema y futuras mejoras

A pesar de que CiviLink cumple con los objetivos funcionales planteados, presenta una serie de limitaciones derivadas principalmente del alcance académico del proyecto y de las restricciones de tiempo durante su desarrollo.

En primer lugar, no se ha llevado a cabo una estrategia de testing exhaustiva. Si bien se han realizado pruebas manuales sobre los casos de uso principales, no se dispone de una batería completa de pruebas unitarias ni de integración que permitan verificar de forma automatizada el correcto funcionamiento del sistema ante cambios futuros. Esta carencia dificulta la detección temprana de errores y limita la capacidad de evolución segura del software.

En segundo lugar, el sistema no ha sido validado frente a escenarios de concurrencia real. En situaciones donde múltiples usuarios acceden simultáneamente a recursos compartidos (como el stock de productos en la cafetería), podrían producirse inconsistencias si no se implementan mecanismos adecuados de sincronización o control transaccional más avanzado.

Asimismo, la aplicación carece actualmente de un sistema de autenticación y control de acceso, lo que implica que no existe diferenciación entre tipos de usuario ni protección frente a accesos no autorizados. Esto limita su aplicabilidad en entornos reales.

De cara a futuras mejoras, se plantea la incorporación de frameworks más modernos que simplifiquen el desarrollo y mantenimiento del sistema, como tecnologías basadas en arquitectura web. Gracias a la modularidad del diseño actual, sería posible sustituir componentes sin afectar significativamente al resto del sistema. Además, la introducción de testing automatizado, control de concurrencia y mecanismos de seguridad permitiría evolucionar CiviLink hacia una solución más robusta y realista para un entorno profesional real.

7. Repositorios del proyecto

- Repositorio de modelado del proyecto: [Modelado CiviLink](#)
- Repositorio de código del proyecto: [Código CiviLink](#)